

Change Impact Analysis Skill

A deterministic, graph-driven deployment-risk engine for AI agents — given a set of changed files, it computes the blast radius, validates API contracts, maps ownership, and scores deployment risk 0–100. No Anthropic API key required.

Version 1.0.0
skill/change-impact-analysis/

Contents

1. Overview
2. Requirements
3. Installation
4. Quick Usage
5. Command-Line Reference
6. Output Files Reference
7. Risk Scoring Reference
8. How the Pipeline Works
9. Repository Layout
10. Troubleshooting

1. Overview

The Change Impact Analysis Skill is a polyglot agent skill for the Deployment / Release phase of the SDLC. Given a set of changed files (explicit, or auto-detected via git), it builds a directed dependency graph, reverse-BFS traverses it to find every directly and transitively affected module, validates API contracts for breaking changes, maps files to owners, and computes a deterministic 0–100 deployment risk score. The host AI agent (Claude Code, GitHub Copilot, or Cursor) reads the engine's output and adds release judgment on top — no separate LLM API key or network call is required by the engine itself.

What you get, every run

#	Output	What it covers
1	Impact Report	Full structured analysis — impacted modules, APIs, regression areas
2	Risk Score	0–100 deployment risk score: LOW / MEDIUM / HIGH / CRITICAL
3	Required Tests	Per-module test suite recommendations
4	Consumer Apps	Downstream applications in the blast radius
5	API Violations	OpenAPI / GraphQL / Protobuf breaking changes detected
6	Deploy Checklist	Ready-to-use pre/post deployment sign-off checklist

Why it exists

Code review catches bugs. Test suites catch regressions. But neither tells you which modules are transitively affected by your change (N hops away in the import graph), whether any API contracts are broken for downstream consumers, or what the deployment risk score is — a single number your release manager can act on. This skill is deterministic (same inputs always produce the same outputs), explainable (every affected file has a traceable import-chain path back to the change), and fast (sub-second BFS on an in-memory graph, even at 100k files).

2. Requirements

Tool	Version	Required
Python	3.8 or later	Yes
Git	any, on PATH	Recommended — enables --from-git; not required with --changed-files
PyYAML	6.0+	Optional — enables YAML-format OpenAPI spec parsing

Without PyYAML, the skill still works fully — it just skips YAML-format OpenAPI specs (JSON specs are always parsed).

3. Installation

Copy skill/change-impact-analysis/ from this repository into your project's skills folder. The same folder contents work for all three tools — only the parent directory changes:

```
your-project/
  .claude/skills/change-impact-analysis/ (Claude Code)
  .github/skills/change-impact-analysis/ (GitHub Copilot)
  .cursor/skills/change-impact-analysis/ (Cursor, + a .cursor/rules/*.mdc)
    SKILL.md
    manifest.yaml
    scripts/
    assets/
    references/
    templates/
```

Verify Python

```
python --version
# Should print Python 3.8 or higher
```

(Optional) install PyYAML

```
pip install -r .claude/skills/change-impact-analysis/assets/requirements.txt
```

4. Quick Usage

Ask your AI assistant in plain language:

```
Analyse the change impact for this PR
What is the deployment risk score for my changes?
Is it safe to deploy?
```

The assistant asks how to identify changed files (auto-detect from git is the default) and where to save output, runs the Python engine, and provides AI-quality narrative — blast radius, contract assessment, and a release recommendation.

Output location options

Option	Flag	Behavior
1 (default)	(none)	./change-impact-output/
2	--output .	Directly in the current directory
3	--output <path>	A specific path you provide

5. Command-Line Reference

```
python skill/change-impact-analysis/scripts/change_impact_skill.py \
  [--repo-path <path>] [--changed-files FILE [FILE ...]] [--from-git]
  [--base-branch <name>] [--since-minutes <N>] [--output <path>]
  [--format markdown|json|both] [--json-only]
```

Argument	Default	Meaning
--repo-path	.	Root of the repository to analyze
--changed-files	—	Explicit list of changed file paths
--from-git	off	Auto-detect changed files from git diff --name-only <base>
--base-branch	main	Base branch for the git diff
--since-minutes	1440 (24h)	Fallback window for the filesystem mtime scan
--output	<repo>/change-impact-output /	Output directory
--format	both	markdown, json, or both
--json-only	off	Emit JSON to stdout, write nothing to disk

Examples

```
# Auto-detect from git
python scripts/change_impact_skill.py --from-git --base-branch main

# Explicit file list
python scripts/change_impact_skill.py \
  --changed-files src/api/users.py src/models/user.py

# JSON only -- pipe into CI gates
python scripts/change_impact_skill.py --from-git --json-only | jq '.risk.score'
```

CI/CD gate example (GitHub Actions)

```
- name: Change Impact Gate
  run: |
    SCORE=$(python scripts/change_impact_skill.py --from-git --json-only \
      | python -c "import sys,json; print(json.load(sys.stdin)['risk']['score'])")
    if [ "$SCORE" -gt "80" ]; then
      echo "CRITICAL risk ($SCORE/100) -- deployment blocked"; exit 1
    fi
```


6. Output Files Reference

File	Description
impact_report.md	Primary artifact — executive summary, risk score, changed/impacted files, regression areas, required tests, contract violations
impact_analysis.json	Machine-readable result for CI/CD integration
deployment_checklist.md	Pre/post deploy sign-off checklist

impact_analysis.json schema

```
{
  "change_detection_method": "git|mtime|explicit",
  "impact": {
    "changed_files": [...],
    "impacted_modules": [{"path": "..", "module_type": "..",
      "change_proximity": "direct|transitive",
      "owners": [...], "risk_factors": [...]}],
    "impacted_apis": [...], "regression_areas": [...],
    "required_test_suites": [...], "consumer_apps": [...],
    "direct_impact_count": N, "transitive_impact_count": N
  },
  "contract_violations": [...],
  "risk": {"score": 0-100, "level": "LOW|MEDIUM|HIGH|CRITICAL",
    "action": "...", "factors": [{"factor": "..", "points": "..", "detail": "..}]}
}
```

A real, unedited example of all three files is in `marketplace/SampleOutputs/sample-app-report/`.

7. Risk Scoring Reference

The deployment risk score is a deterministic 0–100 integer computed from six independent, additive factors, each with a fixed maximum contribution.

Level	Score range	Recommended action
LOW	0–30	Standard release pipeline. No additional gate.
MEDIUM	31–60	Deploy with monitoring. Notify on-call team.
HIGH	61–80	Senior engineer review + rollback plan required.
CRITICAL	81–100	Block deployment. Tech lead escalation + full regression.

Factor breakdown

Factor	Max pts	Drivers
Change volume	20	1–5 files = 4, 6–10 = 8, 11–20 = 14, >20 = 20
Transitive spread	20	0–10 modules = 0–5, 11–20 = 8, 21–50 = 14, >50 = 20
API contract violations	25	Each breaking change up to 10 pts, capped at 25
Module type risk	20	Database +10, API endpoints +6, config/infra +4 (cumulative)
Sensitivity	10	Auth/security code +5, payment/billing code +5
Consumer breadth	5	1 point per distinct consumer app, capped at 5

Full formula, rationale, and worked examples: [references/risk-scoring.md](#).

8. How the Pipeline Works

```
change_impact_skill.py (CLI entry point)
|
[1] GraphBuilder      -- AST import parsing -> directed dependency graph
[2] OwnershipParser   -- maps files to owners via CODEOWNERS/package.json
[3] ContractValidator -- OpenAPI/GraphQL/Protobuf breaking-change checks
[4] ImpactAnalyzer    -- reverse-BFS -> direct + transitive blast radius
[5] RiskScorer        -- deterministic 0-100 score, six weighted factors
[6] ReportGenerator   -- renders impact_report.md + JSON + checklist
```

Every stage is pure Python standard library, deterministic, and runs entirely offline — there is no LLM call anywhere in the engine. The host AI agent only reads the final output and adds narrative. See [Architecture.png](#) for the visual pipeline diagram.

No-Git fallback chain

#	Method	change_detection_method	Fires when
1	git diff --name-only <base>...HEAD	git	Normal PR-style diff against the base branch
2	git diff --name-only HEAD + untracked	git	No diff vs. base — uses uncommitted/staged/new changes
3	git diff --name-only HEAD~1 HEAD	git	Working tree clean — uses the most recent commit
4	Filesystem mtime scan	mtime	No .git folder, Git not installed, or 1–3 found nothing
5	None — exits early	—	Mtime scan also found nothing; re-run with --changed-files

9. Repository Layout

```
change-impact-analysis-skill/  
  skill/  
    change-impact-analysis/  <- copy this into .claude/skills/  
      SKILL.md  
      manifest.yaml  
      scripts/  
        change_impact_skill.py  
      engine/  
      assets/  
      references/  
      templates/  
  marketplace/              <- this folder - docs, guide, samples  
    README.md  
    QuickStart.md  
    UserGuide.pdf            (this document)  
    Architecture.png  
    SamplePrompts.md  
    SampleOutputs/
```

10. Troubleshooting

Symptom	Likely cause	Fix
"python: command not found"	Python 3.8+ not installed or not on PATH	Install Python 3.8+ and re-open your terminal / editor
YAML OpenAPI specs skipped	PyYAML not installed	pip install PyYAML, or ignore if your specs are JSON
change_detection_method is "mtime" unexpectedly	No usable git diff was found	Confirm you're on the right branch with commits ahead of base, or pass --changed-files explicitly
Report says "no files at all" and exits	Both git and mtime fallback found nothing changed	Re-run with an explicit --changed-files list
Risk score seems low for a big change	Some changed files aren't recognized as database/API/config types	Check module_type in impact_analysis.json; path-based heuristics may need the file in a conventional folder name
Owners column is empty	No CODEOWNERS or package.json found, or file not matched by any pattern	Add/update CODEOWNERS at the repo root

If Python or the script is unavailable, the host AI agent falls back to a manual workflow described in SKILL.md's "Manual Fallback" section — it uses `grep_search` for import statements and `references/risk-scoring.md` to compute the score by hand.